



UNIVERSITY OF
TORONTO

The Edward S. Rogers Sr. Department of Electrical & Computer Engineering

Summer Internship Final Report

Migration of a 100Gbps Firewall Accelerator from Intel FPGAs to AMD FPGAs

Gabriel Sebastian Oña Chuquimarca

Advisor: Prof. Paul Chow

14-01-2025

Introduction

Intrusion Detection and Prevention (IDS/IPS) algorithms are fundamental components of data center infrastructure, ensuring cybersecurity and system isolation. While several algorithms have been developed for this purpose, they are primarily implemented using a CPU-only server approach. To overcome this limitation, accelerating IDS/IPS using a dedicated FPGA-based approach offers significant advantages.

To achieve this, the Pigasus Project—a 100Gbps firewall accelerator originally developed by Prof. James Hoe at Carnegie Mellon University—offloads common pattern-matching tasks from the CPU to the FPGA fabric. This approach dramatically enhances performance, enabling faster and more efficient detection of common patterns. It achieves performance improvements by several orders of magnitude, with a single FPGA connected via PCIe to a single CPU capable of replacing over 10 server CPUs.

The repository for this project was designed to run on an Intel Stratix 10 FPGA using the Quartus Prime Pro interface. However, it is incompatible with AMD's Vivado environment. Migrating the RTL hardware design required addressing conflicts related to Altera primitive instances, which presented major challenges. After resolving these conflicts, the next step involved integrating all modules in a compatible manner, adding the necessary wrappers for AMD's primitive equivalents, and ensuring that the TOP.sv module was synthesizable. Finally, a simulation testbench was employed to validate the behavior of the RTL implementation, although additional work remains to be done.

Pigasus Original Repository:

<https://github.com/crossroadsfpga/pigasus>

Pigasus Xilinx Repository:

https://github.com/Gabeinstein/UofT-HPRC_PigasusXilinx/tree/summer2024

Initial State

Previous summer students attempted to migrate this project, but several RTL conflicts arose, primarily related to BRAM memory utilization and Altera primitives. For example, in the file `rom_2port.v`, there are several calls to the `altera_syncram` component module, which is dedicated exclusively to Intel's FPGAs. Another significant example is `altera_dcfifo_synchronizer_bundle.v`, which calls the `altera_std_synchronizer_nocut` module, leading to conflicts during RTL validation. In overview, the complete Pigasus project consists of over 114 Verilog files, along with some extra files generated by a shell script, which introduce additional conflicts when Quartus Prime Pro licenses are not installed on the system.

Previous Knowledge

To address this project, it is necessary to have a basic understanding of some theoretical concepts on networking, hardware/software design, confidence with the Vivado interface, and basic knowledge with Quartus Prime Pro interface.

- **Networking**
 - TCP/UDP bit distribution, debug bridge IP understanding for FPGA remote debug and have a general idea of how the overall system works.
 - It is also crucial to read the paper presenting the Pigasus Project, as it details the architectural choices made during its development.
- **Quartus Prime Pro**
 - Basic knowledge of RTL schematic analysis.
 - Knowledge on how to modify the hierarchy of file modules.
 - Experience with simulation and debugging on MODELSIM.
- **Vivado**

- Confidence with System Verilog, Verilog, Verilog Headers, data structures, imports, synthesis and simulations.
- RTL schematic analysis.
- Knowledge of effective debugging of large-scale RTL testbenches with over half a million nets.
- **Scripting**
 - Basic Linux commands: Specially “grep”.
 - “SSH” communication commands from the agent to the host server machine which has a PCIe connection with the FPGA.

In the lab servers, there is an agent with access to the Quartus Prime Pro interface. It is important to build the complete project in this agent first, before migrating to Vivado, because the repository does not include all the required Verilog files. Some of them are generated with automatic python, shell and TCL scripts that build the entire system.

System Description

The following section focuses on a brief overview of Pigasus project, with a high-level description of each module’s purpose and some key aspects to have in mind.

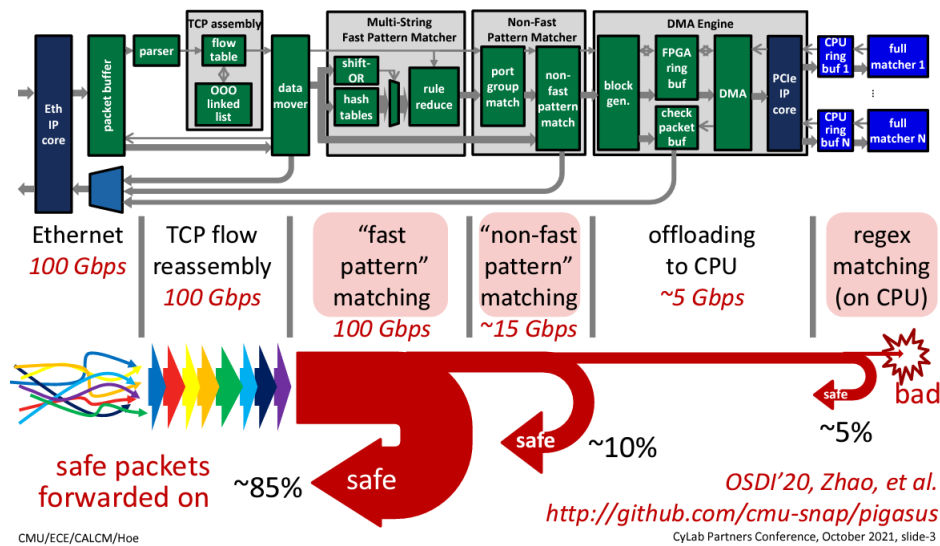


Figure 1. Top Level Pigasus Architecture. Zhao Zhipeng, et al., (2020)

- Ethernet IP core
 - It refers to Intel Stratix 10 FPGA Ips, specifically the CMAC block and all Ethernet hardware configurations to accept incoming packets into the FPGA.
 - It is not essential to understand its implementation if there is no previous knowledge on building IP block designs.
- Ethernet
 - In hardware it is implemented an ethernet service that work as a MUX between some modules. It takes packets analyzed on some stages that pass the tests and return them to the network.
 - Additionally, some FIFOs decouple stages, which in the diagram are represented as a packet buffer.
- TCP Flow Reassembly
 - In this stage, all TCP headers are analyzed and ordered looking at some tables in a linked list.

- This module and its child have a lot of dependencies with FIFOs instances, memory instances, and other calls that caused RTL incompatibility.
- After ordering the packets, they are sent to the data mover to begin pattern matching tasks or send packets back to the network.
- Fast Pattern Matching
 - This module is a service which checks several rules in IDS/IPS with several hash tables and other elements.
 - To debug such a complex module, the project incorporates a file called stats_cnt which is a data structure that handles register files in each module.
 - This module also depends on FIFOs, memory calls and ROM modules.
 - For the rules, there are calls to “.mem” files for bitmaps, hashtables, and rules.
- Non-Fast Pattern Matching
 - Other elements to analyze different rules on IDS/IPS but in this case the processing frequency is slower than the previous module.
 - It has as well bypass services that take successfully safe packets and send them back to the network.
- DMA to CPU
 - Finally, if all other rules are still not satisfied, the Non-Fast Pattern Matching unit sends the remaining packets to be analyzed by a CPU through DMA.
 - The DMA interacts with the PCIe IP core to offload these packets to the host.
- CPU
 - The CPU analyzes the remaining packets and finally sends them back to the network or deletes them if malicious intrusion is detected.

Methodology

- **Building Pigasus Project in Quartus**

As a first step, I cloned the [Pigasus GitHub](#) repository onto a server agent capable of working with Intel Quartus Prime Pro. Next, we need to run the shell script named “*run_pigasus.sh*”. This script will generate the entire project, including additional RTL instances, TCL scripts, VER memory files, and the rest of the hardware components in Quartus Prime Pro.

The project is in the following path:

```
./pigasus/hardware/quartus_project/hardware_test_design/alt_ehipc2_hw
```

What is crucial during this step is to verify the RTL schematic generated for different modules, as their organization and connections must align with what will be used later in Vivado.

After identifying that the Altera primitives were causing RTL issues, I decided to analyze each module, starting from *top.sv*, and review the code for all module instances. In total, there were approximately 114 files in the entire project.

To better understand the relationships between the modules, I created a graph visualization using a Python notebook and highlighted the modules causing conflicts. This approach allowed me to first resolve the primitives that were causing issues and then work backward through the child/parent hierarchy. By doing so, I ensured that each wrapper was both RTL-generatable and synthesizable.

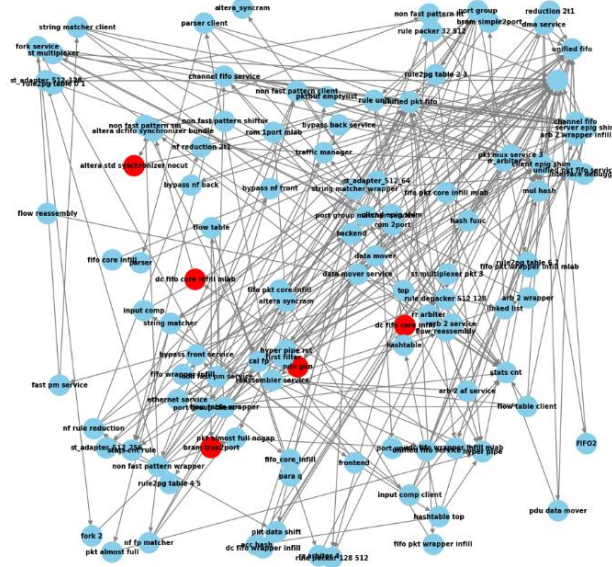


Figure 2. Graph Visualization of RTL Conflicts

Additionally, this tool helped me identify the module with the most instances and understand its relationships with both successors and predecessors.

Top 5 nodes highest degree nodes: [('"', 31), ('unified fifo', 18), ('client epig shim', 17), ('server epig shim', 16), ('unified pkt fifo', 14), ('port group', 13)]

Nodes connected to 'unified fifo' Predecessors: ['dma service', 'unified fifo service', 'channel fifo', 'pdu data mover', 'flow table wrapper', 'arb 2 wrapper infill', 'arb 2 wrapper', 'string matcher wrapper', 'non fast pattern wrapper', 'string matcher', 'port group', 'nf rule reduction', 'linked list', 'nf reduction 2t1', 'reduction 2t1']

Nodes connected to 'unified fifo' Successors: ['dc fifo wrapper infill', 'dc fifo wrapper infill mlab', 'fifo wrapper infill']

Finally, I created a visualization of the main services instantiated in the top-level view, aiming to resolve issues in each one and ultimately integrate them together.

This file is in my [GitHub repository](#) on Python Scripts.

- **Vivado Individual Module Reassembly**

I built individual Vivado projects for each service instantiated in `top.sv` to perform testing and verify that the RTL schematic and synthesis analysis worked as expected. In cases where some modules required memory files, I had to convert their format to `.MEM` for compatibility with Vivado. There are over 35 memory bitmaps, hashes and other files, originally in `.VER` format. To automate this process, I created a Python script that takes a folder, extracts the memory information, and stores it in the required `.MEM` format.

This script is stored in my GitHub repository under the folder *Python Scripts*. I was unable to run the script on an agent due to restrictions with the OS library permissions. It is preferable to run it on an external computer to avoid these agent limitations.

Not all SystemVerilog files are in the `/pigasus/hardware/rtl_sim/src/` folder. Therefore, it is crucial to simultaneously verify the hierarchy tree of your Vivado implementation with the Quartus project.

The main modules that needed modification were:

bram_true2port.sv, bram_simple2port.sv, dc_fifo_wrapper.sv, rom_1port_mlab.sv, rom_2port.sv, and unified_fifo.sv.

To address this, I referred to Xilinx's documentation on RTL generation for these modules, rebuilt them in separate files, and, where necessary, created wrappers to ensure compatibility with the parent module's interface.

In some cases, the header files caused problems regarding its path, so I preferred to have all in the main src directory generated by Vivado. Therefore, it is necessary to delete the path in all the include calls and leave just the name of the header. Finally, the following files should be considered as headers:

Active_msg.vh, channel_if.vh, struct_s.sv, and stats_reg.sv

If it is not assigned as header, then you can change it by going to “Source file properties”, and on “Type” option switch it to Verilog Header.

After doing these processes, for each individual file and reassembling each of the major eight services, the RTL of the top view should be able to generate as well as its synthesis.

RTL Schematic Results

Below I will show the RTL schematic overview of each service.

Reassembler Service

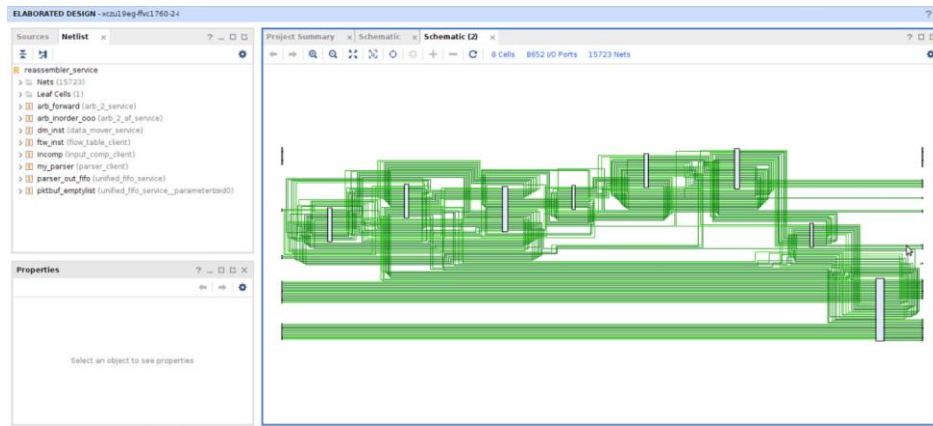


Figure 3. Reassembler Service RTL

Ethernet_service_multi_out

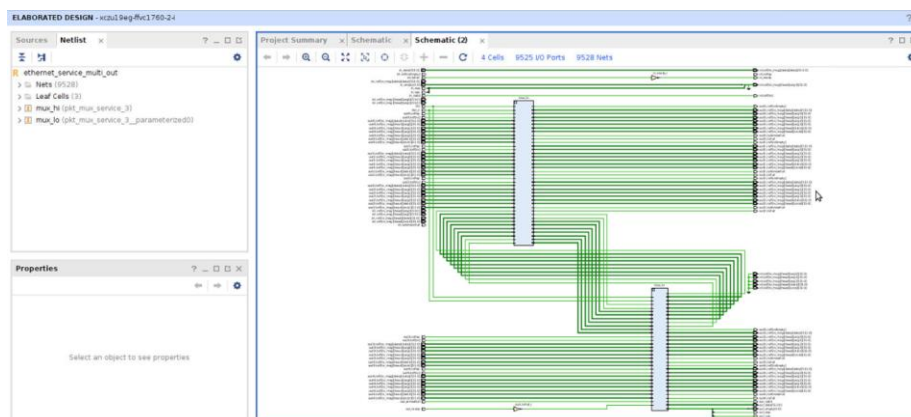


Figure 4. Ethernet Service Multi Out RTL

Fast Pattern Matcher Service

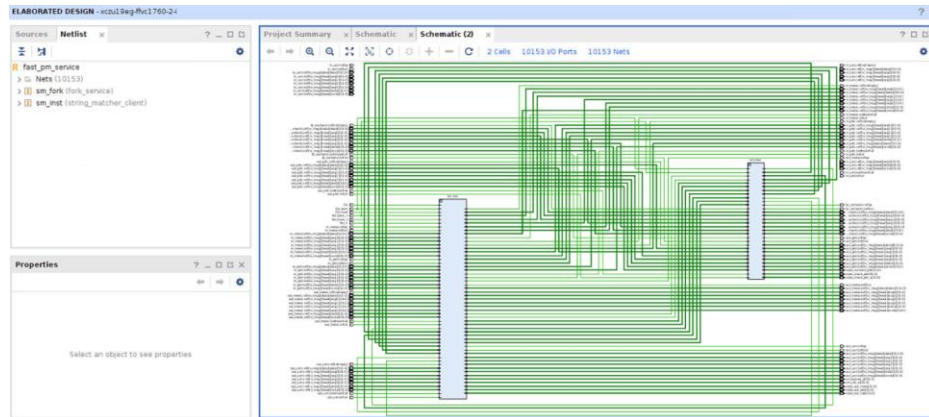


Figure 5. Fast Pattern Matcher Service RTL

Non – Fast Pattern Matcher Service

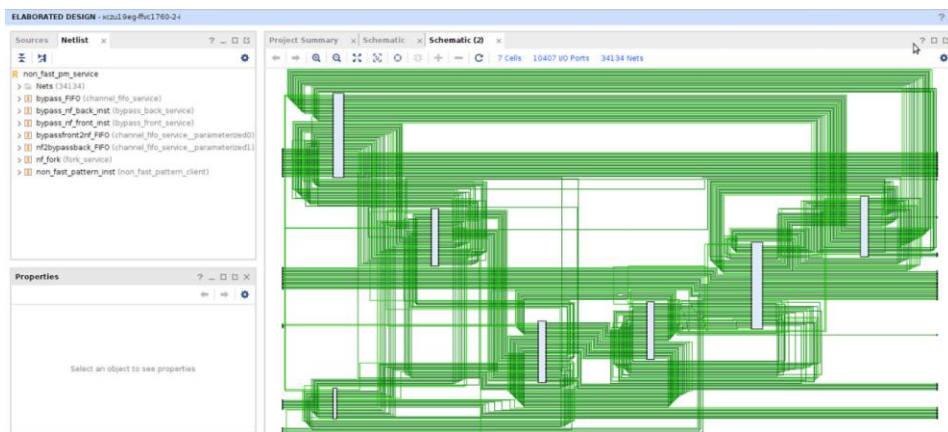


Figure 6. Non – Fast Pattern Matcher Service RTL

Port Group Matcher Service

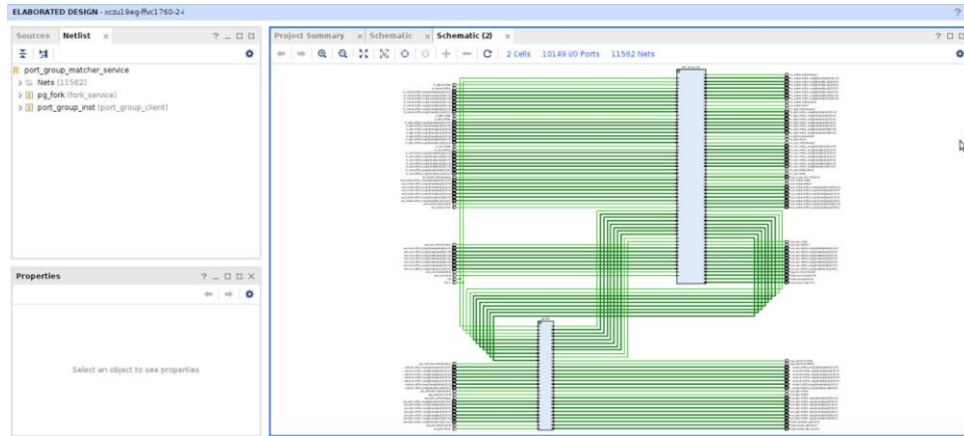


Figure 7. Port Group Matcher Service RTL

Channel Fifo Service

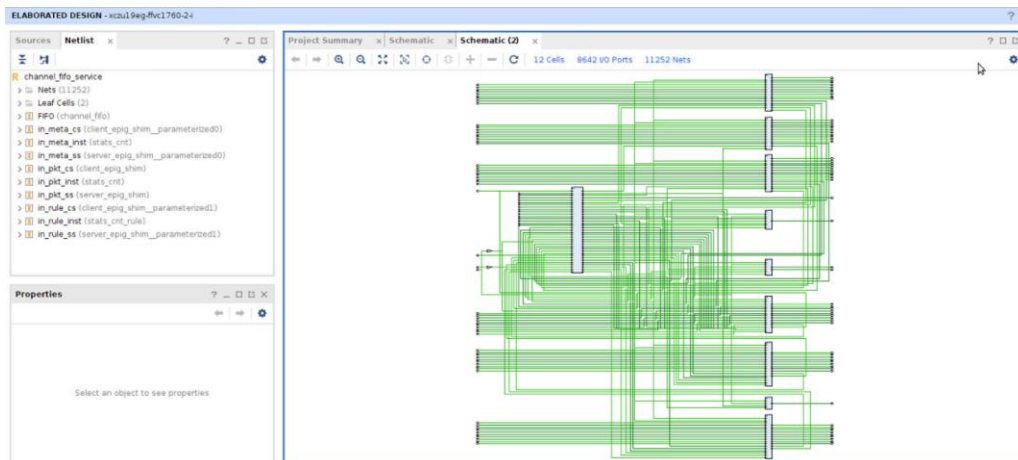
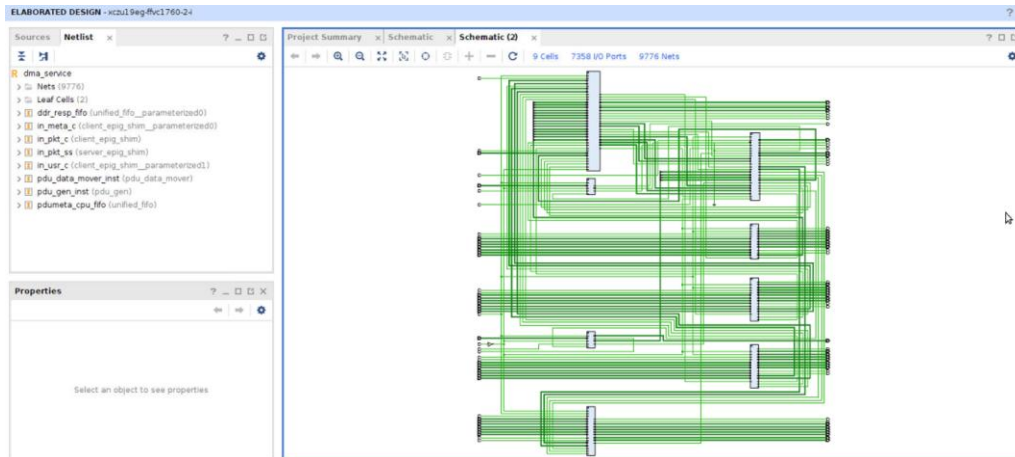


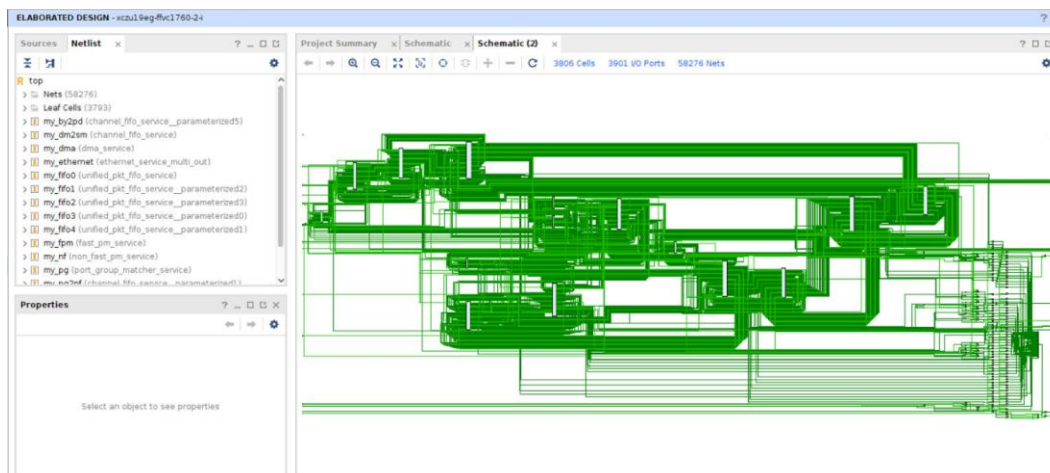
Figure 8. Channel Fifo Service RTL

DMA Service



Once all modules successfully run RTL analysis and synthesis, I joined them together to build the top module.

Top



Another important aspect to mention is that, even though this is the top module, the Pigasus project's testbench incorporates other components to simulate a real working scenario. For instance, in the original testbench, an off-chip ESRAM module was used, which had to be replaced with Xilinx BRAM. Additionally, there is a module in the testbench for simulating PCIe communication to the CPU.

Finally, to debug and better understand various aspects of the data flow, I built a separate project with empty modules connected in the same way as stated in the testbench. I did this to create an RTL-generatable module of the testbench, which I called *testbench_overview.sv*.

Testbench Overview

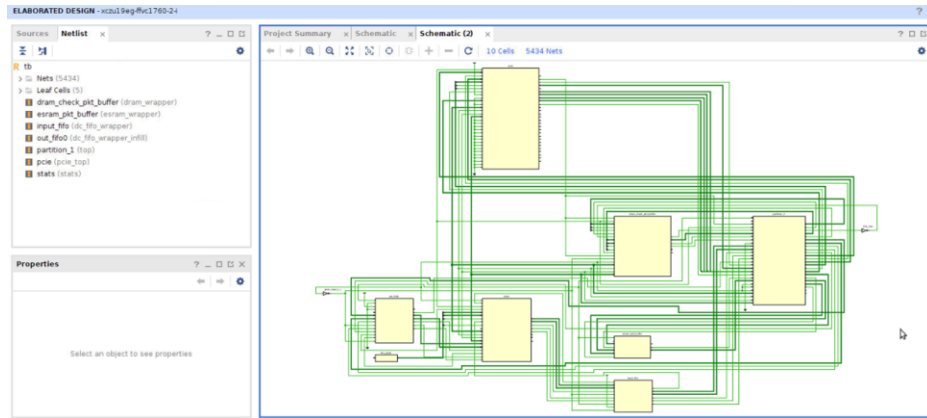


Figure 11. Testbench Overview RTL

Testbench Results

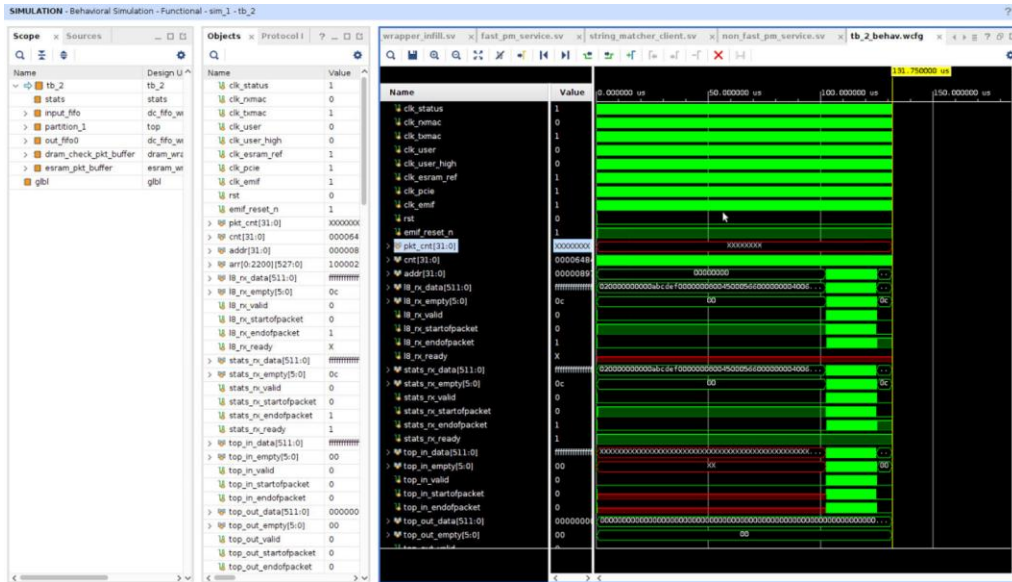


Figure 12. Testbench Simulation

After a quick meeting with Prof. James Hoe, I understood that each packet is pushed and popped by decoupling FIFOs located between two modules. Additionally, counters measure the number of packets processed in each module.

It is important to mention that before packets begin to be delivered through the data stream, the entire project must be properly reset. I attempted to reset in the middle of the simulation, expecting some effect, but nothing happened. The simulation stops once the register *cnt* counts more than 20,000 cycles.

Our registry count results are the following.

```

PKT          64
STOP READING!
REG_IN_PKT      :      100
REG_OUT_PKT     :         0
REG_INCOMP_OUT_META :      100
REG_PARSER_OUT_META :      100
REG_FT_IN_META  :      100
REG_FT_OUT_META :      100
REG_EMPTYLIST_IN :     1067
REG_EMPTYLIST_OUT :      100
REG_DM_IN_META  :         43
REG_DM_OUT_META :         43
REG_DM_IN_FORWARD_META :         0
REG_DM_IN_DROP_META :         0
REG_DM_IN_CHECK_META :         43
REG_DM_IN_000_META :         0
REG_DM_IN_FORWARD_000_META :         0
REG_NOPAYLOAD_PKT :         0
REG_DM_CHECK_PKT :         73
REG_SM_PKT      :         0
REG_SM_META     :         0
REG_SM_RULE     :         0
REG_SM_CHECK_PKT :         0
REG_SM_CHECK_PKT_SOP :         0
REG_SM_NOCHECK_PKT :         0
REG_PG_PKT      :         0
REG_PG_META     :         0
REG_PG_RULE     :         0
REG_PG_CHECK_PKT :         0
REG_PG_CHECK_PKT_SOP :         0
REG_PG_NOCHECK_PKT :         0
REG_BYPASS_PKT  :         0
REG_BYPASS_PKT_SOP :         0
REG_BYPASS_META :         0
REG_BYPASS_RULE :         0
REG_NF_PKT      :         0
REG_NF_META     :         0
REG_NF_RULE     :         0
REG_NF_CHECK_PKT :         0
REG_NF_CHECK_PKT_SOP :         0
REG_NF_NOCHECK_PKT :         0
REG_MERGE_PKT   :         0
REG_MERGE_PKT_SOP :         0
REG_MERGE_PKT   :         0
REG_MERGE_PKT_SOP :         0
REG_MERGE_META :         0
REG_MERGE_RULE :         0
REG_DMA_PKT     :         0
REG_CPU_NOMATCH_PKT :         0
REG_CPU_MATCH_PKT :         0
REG_CTRL        :         0
REG_MAX_DM2SM   :        463
REG_MAX_SM2PG   :         0
REG_MAX_PG2NF   :         0
REG_MAX_BYPASS2NF :         0
REG_MAX_NF2PDU  :         0
REG_SM_BYPASS_AF :       2363
REG_SM_CDC_AF   :         0
-----
Done!

```

Figure 13. Output TCL Results

This is not the expected behavior, and it indicates that not all packets are being popped through the pipeline.

pigasus / pigasus / hardware / rtl_sim / input_gen / example_expected_res			
Zhipeng Zhao init commit of Pigasus 2.0			
Code	Blame	65 lines (63 loc) · 3.17 KB	
1	# REG_IN_PKT	:	100
2	# REG_OUT_PKT	:	90
3	# REG_INCOMP_OUT_META	:	100
4	# REG_PARSER_OUT_META	:	100
5	# REG_FT_IN_META	:	100
6	# REG_FT_OUT_META	:	100
7	# REG_EMPTYLIST_IN	:	1124
8	# REG_EMPTYLIST_OUT	:	100
9	# REG_DM_IN_META	:	100
10	# REG_DM_OUT_META	:	100
11	# REG_DM_IN_FORWARD_META	:	0
12	# REG_DM_IN_DROP_META	:	0
13	# REG_DM_IN_CHECK_META	:	100
14	# REG_DM_IN_OOO_META	:	0
15	# REG_DM_IN_FORWARD_OOO_META	:	0
16	# REG_NOPAYLOAD_PKT	:	0
17	# REG_DM_CHECK_PKT	:	100
18	# REG_SM_PKT	:	100
19	# REG_SM_META	:	10
20	# REG_SM_RULE	:	10
21	# REG_SM_CHECK_PKT	:	10
22	# REG_SM_CHECK_PKT_SOP	:	10
23	# REG_SM_NOCHECK_PKT	:	90
24	# REG_PG_PKT	:	10
25	# REG_PG_META	:	10
26	# REG_PG_RULE	:	10
27	# REG_PG_CHECK_PKT	:	10
28	# REG_PG_CHECK_PKT_SOP	:	10
29	# REG_PG_NOCHECK_PKT	:	0
30	# REG_BYPASS_PKT	:	0
31	# REG_BYPASS_PKT_SOP	:	0
32	# REG_BYPASS_META	:	0
33	# REG_BYPASS_RULE	:	0
34	# REG_NF_PKT	:	10
35	# REG_NF_META	:	10
36	# REG_NF_RULE	:	10
37	# REG_NF_CHECK_PKT	:	10
38	# REG_NF_CHECK_PKT_SOP	:	10
39	# REG_NF_NOCHECK_PKT	:	0
40	# REG_MERGE_PKT	:	10
41	# REG_MERGE_PKT_SOP	:	10
42	# REG_MERGE_META	:	10
43	# REG_MERGE_RULE	:	10
44	# REG_DMA_PKT	:	10
45	# REG_CPU_NOMATCH_PKT	:	10
46	# REG_CPU_MATCH_PKT	:	0
47	# REG_CTRL	:	0
48	# REG_MAX_DM2SM	:	3
49	# REG_MAX_SM2PG	:	11
50	# REG_MAX_PG2NF	:	4
51	# REG_MAX_BYPASS2NF	:	3
52	# REG_MAX_NF2PDU	:	22
53	# REG_SM_BYPASS_AF	:	0
54	# REG_SM_CDC_AF	:	0

Figure 14. Expected Output

Therefore, as Prof. James Hoe mentioned: “the hard part was to bring Pigasus alive, now is a matter of plumbing”, the next step is to check for decoupling FIFOs so that they push and pop all the necessary packets as expected.

When I tried to debug a decoupling FIFO, I observed that all *in_pkt_meta* values are being pushed, but only a few are popped, which could cause issues in the subsequent modules. Additionally, I noticed that some *in_pkt* values—representing the actual body of the TCP packet—are being pushed. However, they should be popped soon after this event, which is not happening as expected.

One possible assumption is that the FIFO is malfunctioning and not popping values at all. However, this is not the case. At *clk* = 10784, I identified an instance where a body packet is pushed and then correctly popped three cycles later, as expected.

Conclusions

- To conclude, dividing the project into separate RTL modules was beneficial for isolating components and working on them individually. This approach made it easier to verify inputs and outputs, analyze the generated RTL schematic, and synthesize the entire module. Furthermore, when integrating all components, it simplified conflict resolution and facilitated the testbench debugging.
- To summarize, using console prints within the testbench helped identify the issue related to decoupling FIFOs. When comparing our output counter results with the original project, we observed that several parts of the system function correctly. However, the FIFO behavior remains unexpected.
- In conclusion, waiting several clock cycles after a reset is crucial for stabilizing elements that are not immediately visible, such as FIFOs, which require time to resume proper operation. Understanding these constraints as well as the packet data flow, demonstrated to be essential for debugging the stream and identifying data stuck in the first decoupling FIFO.

Future Work

- **Decoupling FIFO Issues:** As previously mentioned, the RTL implementation of each individual module appears to be functioning correctly. However, an issue has been identified with the input and output signals of *my_dm2sm* channel FIFO service inside *top.sv*, which do not always behave as expected. This FIFO is responsible for decoupling the *data mover* unit from the *string matcher* unit at the beginning of the dataflow. While it correctly pushes and pops packets in some cases, in others, it only pushes packets without popping them. Prof. James Hoe described this as a "plumbing" task and recommended replacing the channel FIFO service with [Rosebud](#) project FIFOs. Future work should focus on addressing this issue by either implementing the suggested replacement or resolving the signal handshake problems between modules.

- **Ethernet IP Integration:** It will be necessary to connect the RTL Ethernet service from the FPGA fabric to the CMAC 100 FPGA IP block to enable real-time testing. This will help verify whether CPU offloading is functioning correctly. This task is a fundamental first step in integrating the Pigasus Firewall Accelerator into the Galapagos multi-FPGA cluster management system as a separate kernel.
- **DMA Interconnection via PCIe:** Once the above issues are resolved, the next step is to establish a connection to the DMA interconnection through PCIe. This will enable communication with the host CPU, including the final filter in the IDS/IPS verification process.

References

Zhao, Z., Sadok, H., Atre, N., Hoe, J. C., Sekar, V., & Sherry, J. (2020, November). *Achieving 100Gbps intrusion prevention on a single server*. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)* (pp. 1083–1100). USENIX Association.

<https://www.usenix.org/conference/osdi20/presentation/zhao-zhipeng>

Khazraee, M., Forencich, A., Papen, G. C., Snoeren, A. C., & Schulman, A. (2023, March). Rosebud: Making fpga-accelerated middlebox development more pleasant. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (pp. 586-605).

Zhao, Z., Atre, N., Sadok, H., Sahay, S., Obla, S., Hoe, J. C., & Sherry, J. (2022). Pigasus 2.0: making the pigasus IDS robust to attacks and different workloads. In *Proceedings of the SIGCOMM'22 Poster and Demo Sessions* (pp. 61-62).